

Numerical Inverse Kinematics 보고서

로봇 21기 예비단원 최윤서

1. IK의 계산 가능성

1번에서는 역기구학 방정식을 푸는 방법에 대해 얘기해보자.

역기구학 방정식은 비선형 방정식이기 때문에 일반적인 방법(알고리즘)으로 해결할 수 있는 방법이 없다. 그러나 수학 문제를 풀다보면 가끔 1, -1, 0 을 넣으면 우연히 값이 나오는 경우가 생길 수 있다. 이런 식으로 비선형 방정식을 풀 수 있을 때 그 방정식을 폐형식(closed form solution)이라고하고 폐형식의 해를 구하는 방법은 방정식을 직접 푸는 방법이다.

또한, 폐형식으로 풀 수 없는 비선형 방정식은 수치해(numerical solution) 방식을 통해 값을 구해야한다.

수치해 방법은 반복 계산을 통해 근사치를 구하는 방법이다.

수치해는 반복 계산의 특성을 가지므로 계산 과정이 느리다. 그러나 최근 컴퓨터는 연산속도가 빨라져서 큰 영향은 없다고한다.

본 보고서에서는 수치해를 구하는 방법에 대해 설명하고

추가로 singularity가 생겼을 때 예외처리하는 방법까지 확장하고 마무리하고자한다.

2. inverse kinematics 종류

1) 기하학적 방법

로봇의 기하학적 구조를 기반으로 삼각함수나 행렬 연산을 통해 해석적 해를 구함
특정 로봇 구조에서만 사용 가능하고 수학적 분석으로 정확한 해를 갖고 매우 빠른 방법임
구조가 단순한 로봇에 주로 사용됨

2) 수치-해석적 방법

반복적 계산으로 수렴하며 초기값을 기반으로 근사값을 계산함
대부분의 로봇에서 적용 가능하며 초기값이 적절하면 빠르게 수렴할 수 있음
비선형 방정식 풀이에 적합하며 높은 정확도를 보임

3) 휴리스틱 기법

경험적 규칙과 데이터를 기반으로 복잡한 환경에서 작동함
다중 해를 자연스럽게 탐색하며 특이점 문제를 효과적으로 회피함
휴리스틱 기법이 자코비안을 사용한 수치해석적 기법에 비해 더 빠름

3. 수치해석적방법으로 IK 문제 해결하기

FK는 IK보다 비교적 계산하기 쉽기 때문에 θ 값이 주어졌을 때 end-effector pose를 바로 계산할 수 있다.

따라서 IK를 바로 풀지 않고 FK를 반복 호출해서 답을 찾아가는 전략을 활용한다.

예를들어 목표 end effector 좌표가 (0.5, 0.3)이고
현재 관절 값이 (30도, 20도)라고하자.

현재 관절값으로 FK를 계산하여 현재 end effector의 위치를 쉽게 파악할 수 있고 현재 end effector와 목표 end effector 사이의 오차를 구한 후 자코비안을 이용해서 각 관절을 어느정도 움직여야하는지 계산한다.

그 다음 다시 FK를 사용해서 움직인 후에는 오차가 어떻게 변하는지 확인하는 과정을 반복하며 근사적으로 관절값을 조정해 궁극적으로 목표 end effector까지 옮기는 것을 목표로한다.

왜 IK인데 FK를 계속 사용하는가? 에 대한 답변을 주자면
전체적인 큰 틀에서 봤을 땐 결국 순간순간마다 관절값을 조정해서 목표 end effector까지 이동시킨다는 점에서 보았을 때 IK이고
그 관절값을 조정하는 알고리즘에서 FK가 반복적으로 사용된다고 이해하면 될 것 같다.

이해를 돕기위해해 아래 과정에서 사용될 T에 대해 먼저 정의하고 시작하겠다.

end effector 위치가 (x,y,z)에 있다고 가정하자.
하지만, IK를 푸는 과정에서는 좌표만 중요한 것이 아니라 어느 방향에서 이 좌표를 가리키고 있냐도 상당히 중요하다. 따라서 end effector의 상태를 표현하려면

$$p = \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

위치 와 방향 R이 둘다 필요하다.

이때 정의된 R은 회전 행렬이다.

$$R = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix}$$

이 행렬은 엔드이펙터가 현재 어떤 방향을 향하고 있는지를 나타내준다.

따라서 엔드이펙터의 상태는 (R, p) 라고 표현할 수 있다.
즉, 자세는 위치 + 방향으로 표현되는 것이다.

위치와 방향을 따로 가지고 다니면서 계산하기 불편하기 때문에 이를 하나로 합친 것이 T라고 이해하면 된다.

$$T = \begin{bmatrix} R & p \\ 0 & 1 \end{bmatrix} \quad T = \begin{bmatrix} r_{11} & r_{12} & r_{13} & x \\ r_{21} & r_{22} & r_{23} & y \\ r_{31} & r_{32} & r_{33} & z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

왼쪽 위 3X3 = 방향

오른쪽 위 3X1 = 위치

마지막 행 = 행렬을 계산하기 위한 부분으로 이해하면된다.

이렇게 정의된 T를 동차행렬변환이라고한다.

즉, 엔드이펙터의 위치와 방향을 나타내는 행렬이다.

이제 numerical IK를 해결하는 과정을 수식으로 정확히 설명하도록 하겠다.

1) 초기값 설정

아무 관절값 θ 에서 시작한다. 이 초기값이 최종 결과의 성패를 크게 좌우한다.

만약 시작점이 나쁘다면 해를 찾지 못할 수도 있다.

2) FK로 현재 end effector 자세 계산

Product of Exponentials(POE)를 사용하여

$$T(\theta) = e^{[S_1]\theta_1} e^{[S_2]\theta_2} \dots e^{[S_n]\theta_n} M \text{ 라고 계산할 수 있다.}$$

각 기호의 의미는 아래와 같다.

S = i 번째 관절의 스크류축

θ = i 번째 관절각

M = 모든 관절이 기준 자세일 때의 end effector pose

$T(\theta)$ = 현재 end effector의 pose

3) 현재 자세와 목표 자세의 오차 계산

목표 end effector는 T_{target} 이고 현재 end effector는 T_{fk} 이다.

우리의 목표는 현재에서 목표 end effector까지 얼마나 차이가 나는가? 를 계산해야한다

현재 좌표계에서 목표 좌표계까지의 상대 변환을 아래 사진과 같이 계산할 수 있다.

$$T_{diff} = T_{fk}^{-1} T_{target}$$

왜 목표 - 현재로 오차를 구하지않고 굳이 복잡한 계산을 하는가? 에 대한 의문이 드는 것이 당연하다.

오차를 구할 때 목표 end effector 앞에 T_{fk}^{-1} 를 붙이는 이유는 T가 일반적인 숫자 행렬이 아니기 때문이다.

T는 시작하기 전에 말했듯 위치와 회전을 나타내는 변환이다. 그래서 단순히 빼는 것이 아니라 변환을 합성해서 상대적인 변환을 구해야한다.

현재 자세가 T_{fk} 일 때 그 역변환 T_{fk}^{-1} 을 적용하면 현재 자세를 기준으로 다시 원래 기준으로 돌아온다.

따라서 $T_{fk}^{-1} T_{target}$ 는 현재 엔드이펙터에서 봤을 때 목표 엔드이펙터가 어디에 있는지를 나타내준다.

그런데 여기서 $T_{diff} = \begin{bmatrix} R_{diff} & p_{diff} \\ 0 & 1 \end{bmatrix}$ 는 아직 4X4 행렬이다.

그러나 우리가 원하는 것은 “어떤 값을 변화시켜서 관절을 어떻게 움직일건데?” 에 대한 답변이다. 따라서 행렬 그대로는 자코비안과 바로 연결하기 어렵다.

그래서 이것을 6차원 벡터로 바꿔야하고 그 과정은 4) 에서 다루도록 하겠다.

4) SE(3) 행렬을 Twist로 변환

SE(3)에 대해서 먼저 짚고 넘어가자

SE(3)은 쉽게 말해 3차원 공간에서 위치와 회전을 표현하는 변환들의 수학적 공간이다.

따라서, end effector의 위치와 방향을 나타내는 $T(\theta)$ 도 SE(3)에 포함된다.

이제 SE(3) 행렬을 Twist로 변환하는 방법에 대해 알아보자.

Twist는 아래 사진과 같은 6개의 값을 가진다.

$$V = \begin{bmatrix} \omega_x \\ \omega_y \\ \omega_z \\ v_x \\ v_y \\ v_z \end{bmatrix}$$

위에서 3개는 회전과 관련된 성분 / 아래 3개는 이동과 관련된 성분이다.

3)에서 말했듯 4X4 행렬은 현재 end effector에서 목표까지 어떻게 각도를 조절해야하는지 정확한

값이 나와있지 않으므로 T_{diff} 를 Twist로 바꾸는 과정을 거쳐야한다.

이때 log를 사용하면된다.

$$V = \log(T_{diff})^\vee \text{ 를 구하면 된다.}$$

로봇 공학에서 V 에 exponential을 취하면 변환 행렬이 만들어지는 공식이 존재한다. $T = e^{[\hat{V}]}$

우리는 지금 4x4배열인 T를 6X6인 배열 V 로 나타내고싶으니까 위의 공식을 log 형식으로 바꿔서 V 를 T로 표현하고자 한것이다.

여기서 한가지 의문이 더 들 수 있다.

“Twist로 바꿔줘야하는 것까진 이해되는데 왜 굳이 Twist여야하지?” 라는 의문에 답변하자면

Jacobian은 $V = J(\theta)\dot{\theta}$ 라는 관계를 가지고 있다. 이 식은 “관절이 어떤 속도로 움직이면 end effector가 어떤 속도로 움직이는가?” 를 의미한다. 결국 jacobian을 계산해야하는데 그 때 Twist가 쓰이기 때문에 이 과정에서 T를 Twist로 변환하는 과정을 거친 것이다.

그러면 다음으로 jacobian의 입력과 출력에 대해 알아보겠다.

5) jacobian의 입력과 출력

jacobian $V = J(\theta)\dot{\theta}$ 은 $\dot{\theta}$ 를 입력했을 때 J 가 변환해서 end effector의 속도 V 를 만들어줌

즉, $\dot{\theta} \xrightarrow{J} V$ 의 과정을 거치는 것이다.

우리는 현재 4) 과정을 통해서 V 를 이미 알고있다.

그렇다면 우리가 원하는 것은 $\dot{\theta}$ 이다.

즉, “end effector 움직임을 만드려면 관절을 어떻게 움직여야하지?” 라는 문제만 해결하면 IK문제를 풀 수 있다.

이를 단순히 생각해서 $\dot{\theta} = J^{-1}V$ 라고 바꿔서 쓸 수 있다면 좋겠지만 아쉽게도 Jacobian은 역행렬이 존재하지 않는다.

따라서 Jacobian의 유사역행렬인 $J^\dagger$ 를 사용한다. 그러면 이렇게 나타낼 수 있다. $\dot{\theta} = J^\dagger V$

$\dot{\theta} = J^\dagger V$ 를 이용해서 관절을 수정하면 $\theta^{k+1} = \theta^k + \alpha J^\dagger V$ 라고 간단하게 표현할 수 있다.

이때 α 는 얼마나 크게 수정할지를 결정하는 값이다.

6) FK를 다시 계산

5) 과정에서 새로운 관절값 θ^{k+1} 을 얻을 수 있었다.

그러면 다시 처음으로 돌아가서 새로운 관절값으로 실제 end effector가 어디에 있는지를 다시 계산해야한다.

$T_{diff} = (T_{fk}^{k+1})^{-1}T_{target}$ 이 식을 다시 계산하고 1)부터 6)까지의 과정을 동일하게 반복하면서 관절값을 계속 목표치와 조금씩 가깝게 만들면서 근사치로 일치할 때까지 이 과정을 반복하면

결론적으로 numerical IK 문제를 해결할 수 있다.

4. singularity 예외 처리

Singularity란 무엇인가부터 얘기하고 넘어가는 것이 좋을 것 같다.

IK문제 푸는 과정의 5)번 과정에서 Jacobian의 역행렬이 존재하지 않기 때문에 유사역행렬을 사용해서 $\dot{\theta} = J^\dagger V$ 라고 식을 풀었다.

이때 만약 현재 로봇 자세가 특이점 근처라서 $J^\dagger$ 을 그대로 사용하면 관절값이 비정상적으로 커지는 현상을 Singularity라고한다.

Singularity 현상이 발생하게 되면 작은 end effector 오차를 해결하려고해도 갑자기 오차가 수십~수백배 수준으로 커질 수 있다. 따라서 numerical IK에서 Singularity 예외처리는 필수이다.

예외처리 방법에 대해 설명하겠다.

1) Singular Value 확인하기

$J = U\Sigma V^T$ 를 계산한 다음 가장 작은 Singular Value를 확인한다.

이때 Singular Value를 $\sigma_{\min}$ 라고 하는데 이 최솟값이 너무 작으면

즉, $\sigma_{\min} < \epsilon$ 인 상태라면 특이점 또는 특이점 상태라고 판단한다.

이때 엡실론 값은 본인이 원하는 최소치로 사용자가 직접 코드에서 설정해주면 된다.

2) Damped Least Squares

이 방식은 실제로 IK에서 가장 많이 사용하는 방식이다.

일반적인 유사역행렬을 사용하는 것이 아니라

$$J^\dagger = J^T (J J^T + \lambda^2 I)^{-1}$$

위 사진처럼 유사역행렬을 대체해서 사용하는 방식이다.

여기서 λ 가 damping factor이고 특이점 근처에서 λ 를 크게 만들어서 관절값이 폭발하는 것을 막는다.

왜 damping이 효과가 있는가?

$$\frac{1}{\sigma}$$

일반적인 유사역행렬에서는 $\frac{1}{\sigma}$ 을 사용하는데 Damped Least Squares에서는

$\frac{\sigma}{\sigma^2 + \lambda^2}$ 의 형태를 사용한다. 따라서 σ 가 너무 작아져서 관절값이 갑자기 튀려고해도

λ 가 그에 비례해서 값을 다시 잡아주기 때문에 관절값이 매우 커지지는 않는다.